

Exam Notes

Date:

23K0055

Databases Assignment 4

BAE 5-A

(Q1a) Atomicity is violated. It requires that either all operations of a transaction happen or none happen. Wallet was charged, but booking was not completed. This is a partial transaction, which directly violates atomicity.

Consistency has likely been violated. It ensures that database moves from one valid state to another. After the crash the customer wallet shows deductions and there is no book entry that exists. This is an invalid state.

Isolation is not validated. No interference from concurrent transactions

Durability depends on the timing. If wallet debit was written to non-volatile storage before the crash, it is durable, but the booking was not made durable. Durability is partially applied, which still leads to inconsistency.

(Q1b) The failure occurred due to commit of the booking not being completed after the system wrote the wallet deduction to the DB. No atomic coordination b/w wallet service & booking service. Application confirmed only one part of the transaction. Server crashed before the commit point of the overall process

Date:

Q1c) Using 2-Phase Commit

- start transaction coordinator
- Wallet service places the deduction in a pending state
- Booking service prepares booking entry
- If both sub-transactions are ready and both agree then commit both. If any fails, rollback both

Q1d) If unresolved, ^{there} ~~there~~ is real damage to brand reputation. Customer trust decreases due to unexplained deductions. There will also be a high load on customer support for refund complaints

Q2a) T1 (Sale) $Q = Q - 5$
T2 (Return) $Q = Q + 2$

1. T1 read_item (100)
2. T2: read_item (100)
3. T1: write_item (95)
4. T2: write_item (102)

Final $Q = 102 \rightarrow$ Incorrect

Date:

Q2b) Precedence graph

$T1 \text{ read_item}(Q) \rightarrow T2 \text{ write_item}(Q) \Rightarrow T1 \rightarrow T2$

$T2 \text{ read_item}(Q) \rightarrow T1 \text{ write_item}(Q) \Rightarrow T2 \rightarrow T1$

Graph contains cycle $T1 \leftrightarrow T2$

↳ not conflict serializable

Q2c) $T1$: read 100 $(T1 \rightarrow T2)$ Serial 1

$T2$: write 95 Final $Q = 97$

$T3$: Read 95

$T4$: Write 97

$(T2 \rightarrow T1)$ Serial 2

Produces the same final $Q = 97$

Q2e) The real world impact is that there are incorrect stock levels shown on website. There are wrong reordering decisions. Customer complaints due to unfulfilled orders

Date:

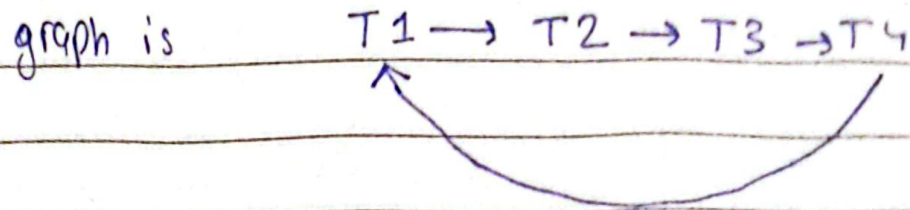
Q3) 1. T1

$T1 \rightarrow T2$ (b)

$T2 \rightarrow T3$ (c)

$T3 \rightarrow T4$ (d)

$T4 \rightarrow T1$ (e)



2. since there is a cycle present, we know that the graph is not serializable.

3. No equivalent serial schedule

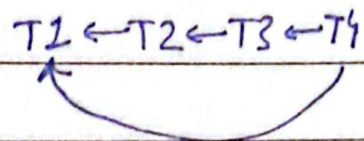
Precedence graph for S2

$T4 \rightarrow T3$ (d)

$T3 \rightarrow T2$ (c)

$T2 \rightarrow T1$ (b)

$T4 \rightarrow T1$ (a)



serializable

equivalent serial schedule for S2 is

$T4 T3 T2 T1$

Date:

Q4) 1) Lost update problem

Both transactions read the same value of 2000. Each computes a new value based on that read & writes it. ~~So~~ T2 overwrites T1 result.

T1's update is lost.

The final total is not correct

2. The final value is 2800 because T1 wrote 1700 and T2 did an overwrite of T1 with 2800.

4. Correct serial outcomes

3. Order A: T1 $\rightarrow$ T2

start: 2000

After T1 (discount): $2000 - 300 = 1700$

After T2 (add product): $1700 + 800 = 2500$

Result (T1 $\rightarrow$ T2): 2500

Order B: T2 $\rightarrow$ T1

Start: 2000

After T2 (add product): $2000 + 800 = 2800$

After T1: $2800 - 300 = 2500$

Result (T2 $\rightarrow$ T1) = 2500

Date:

The correct result would be 2500.

Q5) a) $T1 \rightarrow T2, T1 \rightarrow T3, T2 \rightarrow T3$

equivalent serial schedule is
 $T1 \rightarrow T2 \rightarrow T3$.

b) $T1 \rightarrow T3, T3 \rightarrow T1, T1 \rightarrow T2, T3 \rightarrow T2$.

~~cycle~~ cycle present, not
conflict serializable

c) $T2 \rightarrow T3, T3 \rightarrow T1, T2 \rightarrow T1$ no cycle
so it's conflict serializable.

equivalent serial schedule is

$T2 T3 T1$

d) $T2 \rightarrow T3, T1 \rightarrow T3, T3 \rightarrow T1, T2 \rightarrow T1$

we have $T1 \rightarrow T3 \rightarrow T1$ so it's not
conflict serializable.